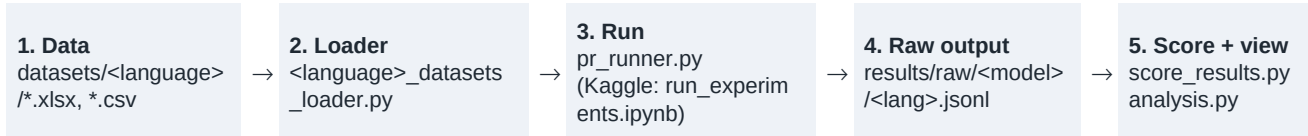


Run Guide: Prompt Repetition in RTL Languages

Which file to run for which language, step by step, and how to view the results · repo
FizzahNasir/prompt_repetition_research

How the pieces fit



Prompts, scenarios and scoring live in **experiment_runner.py** (system prompts: SYSTEM_PROMPT_PA / UR / PS / AR / FA / SD). The task-to-scenario mapping is in **run_punjabi.py**. Run every command from the repository root.

Which file for which language

Code	Language	Loader (run via --languages)	Data files in datasets/<folder>/	Items
pa	Punjabi (Shahmukhi)	punjabi_datasets_loader.py	punjabi/: MGSM_, ARC_, OpenBookQA_, CommonSenseQA_Punjabi.xlsx (Google Translate) + NameIndex_, MiddleMatch_, ScriptMixed_Punjabi.xlsx	1,933
ur	Urdu	urdu_datasets_loader.py	urdu/: MGSM_, ARC_, CommonSenseQA_Urdu_GoogleTranslate.xlsx, openbookqa_urdu_test.csv + NameIndex_, MiddleMatch_, ScriptMixed_Urdu.xlsx	1,945
ps	Pashto	pashto_datasets_loader.py	pashto/: mgsm_pashto_translated.csv, arc_pashto.csv, openbookqa_pashto.csv, COMMONSENSE PASHTO - commonsenseqa_pashto.csv + NameIndex_, MiddleMatch_, ScriptMixed_Pashto.xlsx	1,709
ar	Arabic	arabic_datasets_loader.py (MCQ/MGSM via translated_benchmarks.py)	arabic/: ARC_, OpenBookQA_, CommonSenseQA_, MGSM_Arabic_GoogleTranslate.xlsx + NameIndex_, MiddleMatch_, ScriptMixed_Arabic.xlsx	1,941
fa	Persian	persian_datasets_loader.py (via translated_benchmarks.py)	persian/: ARC_, OpenBookQA_, CommonSenseQA_, MGSM_Persian_NLLB.xlsx + NameIndex_, MiddleMatch_, ScriptMixed_Persian.xlsx	1,932
sd	Sindhi	sindhi_datasets_loader.py (via translated_benchmarks.py)	sindhi/: ARC_, OpenBookQA_, CommonSenseQA_, MGSM_Sindhi_NLLB.xlsx + NameIndex_, MiddleMatch_, ScriptMixed_Sindhi.xlsx	1,922

Each language has the same 7 tasks: GSM8K (MGSM), ARC, OpenBookQA, CommonSenseQA, NameIndex, MiddleMatch, ScriptMixed. For ar/fa/sd, **translated_benchmarks.py** picks the *_NLLB* files automatically when they exist, otherwise *_GoogleTranslate*. Model keys: qwen2.5-1.5b, qwen2.5-3b, qwen2.5-7b, mistral-7b-v0.3, gemma2-2b, llama3.2-1b, llama3.2-3b, llama3.1-8b (Llama and Gemma are gated on Hugging Face).

Step by step

Step 0 · Install

For checking data, scoring and figures (any machine):

```
pip install pandas openpyxl scipy matplotlib huggingface_hub
```

For running models you need an NVIDIA GPU: `pip install vllm==0.30.0` (Linux), or transformers + torch as the slower fallback. Gated models need `huggingface-cli login` or an `HF_TOKEN` environment variable with the licences accepted.

Step 1 · Check that a language loads

Prints the number of items for one language (replace ar with pa, ur, ps, fa or sd):

```
python -c "import pr_runner as p; print(len(p.load_dataset('ar')))"
```

Step 2 · Dry run (count prompts, no GPU)

```
python pr_runner.py --model qwen2.5-3b --languages ar --dry-run
python pr_runner.py --model qwen2.5-3b --dry-run # all 6 languages
```

Step 3 · Smoke test (20 prompts, GPU)

```
python pr_runner.py --model qwen2.5-1.5b --languages ar --limit 20 --out smoke
```

Step 4a · Full run on Kaggle (how the published results were made)

Open notebook `run_experiments.ipynb` (Kaggle: *fiznasir/prompt-repetition-tests-1-2*) in the editor and set the first code cell:

```
MODELS = ["llama3.2-1b", "qwen2.5-3b", ...] # any model keys
LANGUAGES = ["pa", "ur", "ps", "ar", "fa", "sd"]
METHODS = ["baseline", "repetition"]
SKIP = [] # e.g. ["sd:GSM8K"] to leave a task out
```

- Input: *pr-results-checkpoint* on its latest version (so finished work is skipped).
- Add-ons → Secrets: HF_TOKEN (write token) ticked. GPU T4 x2, Internet on.
- **Save Version** → **Save & Run All (Commit)**. Do not use `kaggle kernels push`: it drops the secret.
- Download the output afterwards: `kaggle kernels output fiznasir/prompt-repetition-tests-1-2 -p kaggle_out`

Step 4b · Full run on your own GPU or Colab (one model per command)

Results are appended to *results/raw/<model>/<lang>.jsonl*; re-running the same command skips everything already done.

```
python pr_runner.py --model qwen2.5-3b --out results # all languages
python pr_runner.py --model qwen2.5-3b --languages pa ur --out results
python pr_runner.py --model llama3.1-8b --out results --hf-repo
<hf-user>/prompt-repetition-results
```

Useful options: `--tasks ARC GSM8K`, `--skip sd:GSM8K`, `--shard 0/2` (with `CUDA_VISIBLE_DEVICES=0`, one process per GPU), `--backend hf` (no vLLM).

Step 5 · Get the finished results (instead of re-running)

The final raw generations (all 8 models × 6 languages) are in the Kaggle dataset *fiznasir/pr-results-checkpoint* as flat files *<model>__<lang>.jsonl*. Download and put them back into the *raw/<model>/* layout:

```
kaggle datasets download fiznasir/pr-results-checkpoint -p ckpt --unzip
python -c "import pathlib, shutil; [shutil.copy(f, pathlib.Path('results/raw',
f.name.split('__')[0]).mkdir(parents=True, exist_ok=True) or pathlib.Path('results/raw',
*f.name.split('__'))) for f in pathlib.Path('ckpt').glob('*__*.jsonl')]"
```

Step 6 · Score everything (all languages and models)

Re-scores every stored response and runs the statistics. Prints the wins/losses tables and writes the CSVs:

```
python score_results.py --results results --out results/analysis
```

Output file	What it contains
scored.csv	one row per generation, with <code>is_correct</code> (input for <code>analysis.py</code>)
accuracy_table.csv	accuracy per language × model × task × scenario × method
mcnemar_results.csv	McNemar per test: <code>p_value</code> (paper, chi-square) and <code>p_exact</code>
wins_summary.csv	significant wins / losses per language × model × method

Step 7 · View one language (tables + Figure 1)

analysis.py works on one language at a time. Cut scored.csv down to that language, then render (replace ar with pa, ur, ps, fa or sd):

```
python -c "import pandas as pd; d = pd.read_csv('results/analysis/scored.csv'); d[d.language == 'ar'].to_csv('results/analysis/scored_ar.csv', index=False)"
python analysis.py --csv results/analysis/scored_ar.csv --out results/analysis/ar
```

Writes *accuracy_table.csv*, *mcnemar_results.csv* and **figure1.png** (accuracy bars per task, scenario and method, one panel per model) into *results/analysis/ar/*, and prints the tables. Add `--no-plot` to skip the figure.

Step 8 · Tests 3–4 (cross-lingual methods, later)

Add the two methods; finished baseline / repetition rows are reused, only the new ones run:

```
python pr_runner.py --model qwen2.5-3b --out results \
  --methods baseline repetition cross_lingual_t1 cross_lingual_t2
```

On Kaggle: set METHODS to the same four names in the notebook.

Rebuilding the data (only if you change it)

Warning: regenerating or re-translating changes the items, so existing results no longer match. These were already run for the published results.

```
python generate_retrieval_tasks.py # NameIndex + MiddleMatch, all 6 languages
python generate_retrieval_tasks.py --scriptmixed # also regenerate ScriptMixed
python translate_benchmarks.py --langs ar --backend gtx # Google Translate
python translate_benchmarks.py --langs fa sd --backend nllb --cache nllb_cache.json # NLLB, GPU
python dataset_acquisition.py --all # optional HF source datasets
```

Older scripts (not used for these results)

- **run_punjabi.py**, **run_all_languages.py**: earlier runners (API models via litellm, e.g. `python run_punjabi.py --language ur --models gpt-4o-mini --dry-run; --analysis-only --results <file>` re-analyses their output).
- **colab_run.py**: earlier single-script Colab runner with transformers; use `pr_runner.py` instead.
- **analysis.ipynb** reads `run_punjabi.py` CSVs; **results_analysis.ipynb** is from an older design and is not valid JSON.

Troubleshooting

- **Windows 'charmap' error with kaggle**: set `PYTHONIOENCODING=utf-8` first.
- **Kaggle log says 'No HF_TOKEN secret' or HF mirror (disabled)**: the secret is not attached; stop the run and attach it in Add-ons → Secrets.
- **kaggle datasets version fails with a path error**: run it from a short folder with a relative `-p`. A new dataset version replaces all files, so upload the full set.
- **Gemma-2 is slow** (fp32 on T4, ~2 prompts/s): put it last in MODELS.
- **Git corrupting binary files**: the repo's `.gitattributes` marks pdf/xlsx/zip/images as binary; keep it.

Full state and history of the project: **RESUME.md**. Results summary: *results/tests_1_2/Prompt_Repetition_Tests_1-2_Report.pdf*.